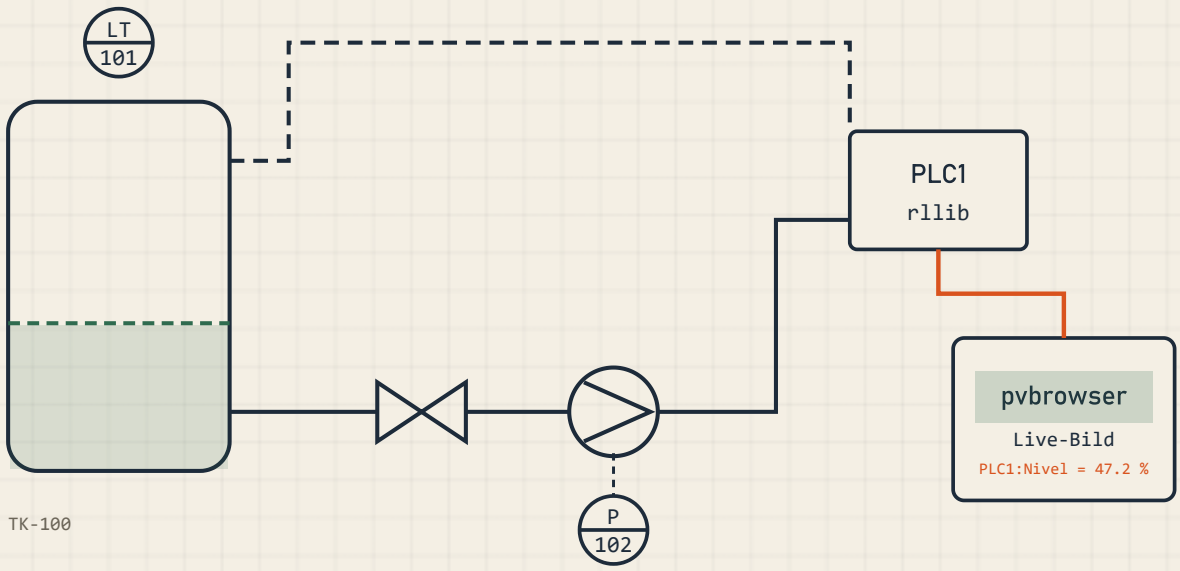
OFFENES SCADA / HMI

PV Suite

FREIE SOFTWARE
FÜR DIE ANLAGE GEMACHT

Bedienbilder, die im Designer entstehen, ohne Umwege an die SPS angebunden werden und unter Windows, Linux und macOS laufen.



vom Sensor zum Bild, ohne Code zu schreiben

VERSION
1.0.0-beta.4

PLATTFORMEN
Windows · Linux · macOS

LIZENZ
GPL / LGPL

Was PV Suite ist

PV Suite baut auf pvbrowser auf, einem freien SCADA mit vielen Jahren Praxis in der Anlage, bringt es auf Qt 6 und ergänzt, was bisher am meisten Handarbeit kostete: die Tags, den Kommunikations-Daemon und die Logik hinter jeder Taste.

Der Ansatz ist einfach: ein Bedienbild, das sich mit der Maus erstellen lässt, eine einzige Stelle, an der die Verbindung zur SPS festgelegt wird, und eigener Code, der weiterhin bereitsteht, wenn es feiner werden muss.

Es ersetzt niemanden, der programmiert; es nimmt ihm das Wiederkehrende ab. Was früher Hunderte Zeilen waren, um ein Register zu lesen, zu skalieren und einzufärben, ist heute eine Zeile in einer Tabelle.

FÜR WEN ES IST

- Integratoren, die Schaltschränke und Visualisierung für kleine und mittlere Kunden bauen.
- Instandhalter, die die Anlage sehen wollen, ohne von teuren Lizenzen abzuhängen.
- Lehrende und Lernende der Automatisierung, die etwas Echtes zum Üben brauchen.
- Alle mit pvbrowser-Projekten, die sie ohne Neuanfang auf Qt 6 bringen wollen.

> *„Ein Bedienbild soll zeigen, was in der Anlage passiert, nicht wie mühsam es zu programmieren war.“*

Die vier Bausteine

PB

PVBROWSER

Der Client: zeigt die Bilder, die der Server schickt. Mit eingebautem Webbrowser.

PS

PVSERVER

Die Bibliothek, mit der die Bild-Server programmiert werden, einer pro Projekt.

PD

PVDEVELOP

Die Werkbank: Editor, Bild-Designer, Tags und Daemon-Generator.

RL

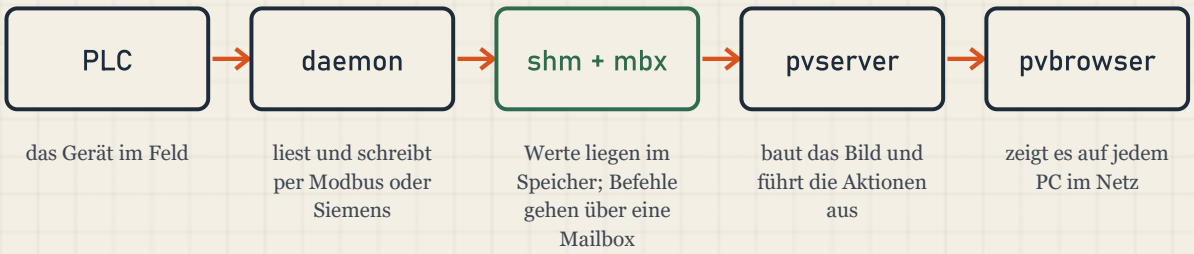
RLLIB

Die Kommunikation: Modbus, Siemens, Shared Memory, Mailboxen und Tags.

02 · IM INNEREN

Wie ein Wert reist

Jeder Baustein macht genau eine Sache, und die richtig. So legt eine Kommunikationsstörung die Bilder nicht lahm, und ein hängendes Bild hält das Lesen der SPS nicht auf.



Der Daemon wird von pvdevelop aus der Tag-Tabelle erzeugt. Er liest die SPS zyklisch, legt jeden Wert mit seiner Qualität im Shared Memory ab und bearbeitet die Schreibaufträge aus der Mailbox.

Der pvserver spricht nie mit der SPS: Er liest diesen Speicher. Steht der Daemon, merkt das Bild es und kennzeichnet den Wert als ungültig, statt eine alte Zahl als aktuell anzuzeigen.

pvbrowser ist schlank: Er empfängt das Bild über das Netz und schickt die Klicks zurück. Mehrere können gleichzeitig offen sein, auf verschiedenen PCs.

OHNE CODE

In den Eigenschaften jedes Widgets legt die Schaltfläche **Tags…** fest, was es tut und wie es aussieht. pvdevelop erzeugt den Code beim Speichern des Bildes. Fährt man mit der Maus über ein Feld, erklärt eine Hilfe, wofür es dient und bei welchen Widgets es wirkt.

AKTIONEN

einen Wert schreiben, umschalten, mit Grenzen addieren oder subtrahieren, eine Zahl eingeben, zu einem anderen Bild wechseln, um Bestätigung bitten, nur bei erfüllter Bedingung handeln.

ANIMATIONEN

formatierter Text, Farbe nach Regeln, Blinken, Sichtbarkeit, Bild nach Wert, Bildfolgen und eine Markierung für ungültige Daten.

```
//pvb: btnBomba press=write:PLC1:Bomba=1
  release=write:PLC1:Bomba=0
color=PLC1:Bomba
```

Was enthalten ist

BENANNTE TAGS

Strukturen, Arrays und Bits von Ganzzahlen (`PLC1:Estado.3`). Einmal festgelegt, in allen Bildern nutzbar.

SIMULATION

Der Daemon startet mit `-sim` gegen einen Modbus-Simulator: alles testen, ohne die SPS daneben.

TAG-MONITOR

Live-Werte mit Qualität. Schreiben und Forcen während der Inbetriebnahme.

ANGENEHMER DESIGNER

Kopieren, Einfügen, Ausrichten und Verteilen; ein Doppelklick öffnet die Eigenschaften.

INTEGRIERTES KOMPILIEREN

Ein Fenster mit markierten Fehlern im Editor. MinGW oder Visual Studio unter Windows, g++ unter Linux, mit dem Qt jedes Projekts aus der Symbolleiste gewählt.

VIER SPRACHEN

pvdevelop und pvbrowser auf Spanisch, Englisch, Portugiesisch und Deutsch, mit Handbuch und integrierter Hilfe.

Erste Schritte

1. In pvdevelop **File > New**: Ordner und Projektname.
2. **Daemon > Modbus**: SPS-Adresse und Tag-Tabelle. Beim Kompilieren entstehen der Daemon und seine `.tagmap` .
3. Im Designer Widgets einfügen und in den Eigenschaften mit **Tags…** Aktionen und Animationen vergeben.
4. **Strg+T** kompiliert und startet den Server; **Strg+P** öffnet pvbrowser.
5. **Daemon > Tag-Monitor** zum Beobachten und Forcen. Das Beispiel `01_modbus` enthält alles fertig.

Installation

Windows 64 Bit: ein Installer, der alles nach `C:\PVSuite` legt. Wer nur beobachten will, installiert pvbrowser allein.

Linux x86-64: eine `.run`-Datei, die nach `/opt/pvb` installiert, Qt inklusive. Fehlt dem Desktop eine Bibliothek, installiert `preparar_linux.sh` sie.

macOS (Apple Silicon): pvbrowser und pvdevelop als Zip, noch Beta und unsigniert.

Ist es schon installiert, bietet der Installer ein Update an: Nichts muss von Hand gelöscht werden.

LIZENZ

pvbrowser und pvdevelop stehen unter der GPL. pvserver und rllib unter der LGPL, daher können Ihre Server jede beliebige Lizenz tragen.

ÄLTERE PROJEKTE

pvbrowser-Projekte laufen weiter. Mit der MinGW-8.1-Komponente lassen sie sich wie bisher bauen, 32 oder 64 Bit, und in Ruhe auf Qt 6 umstellen.

Ein paar Worte

PV Suite ist aus dem ernsthaften Einsatz von pvbrowser entstanden und aus dem Wunsch, dass Mühsames nicht mehr mühsam ist. Jede Funktion kommt aus einem konkreten Bedarf: eine Taste, die ein Register schreiben musste, ein Bild, das nicht meldete, dass der Daemon ausgefallen war.

Es ist freie Software und noch in der Beta: ausprobieren, an Grenzen bringen, verbessern. Wenn etwas nicht wie erwartet funktioniert, melden Sie es auf GitHub; so wird die nächste Version besser.

Néstor Palomeque

WO ES ZU FINDEN IST

QUELLCODE
`github.com/nestorlevi/PvSuite`

INSTALLER
`github.com/nestorlevi/PvSuite/releases`

DAS PROJEKT UNTERSTÜTZEN
`cafecito.app/nestorlevi`
Wenn es Ihnen nützt, können Sie mir einen Kaffee spendieren (Cafecito, Argentinien).

BASIERT AUF
`pvbrowser.de` — Rainer Lehrig